Tarang Pharma Distribution, expense receipts

Done By: Karthika S (rskarthika08@gmail.com)

1. Problem I Found:

Nilesh Agrawal, Tarang's Finance Controller, asked for a tool that reads a bill photo and fills in the vendor, bill date, bill number, GSTIN and amount. He wants at least 95% accuracy on the fields, and he wants claims under ₹2,000 approved straight from the photo.

Accuracy on single fields is not enough. **A claim is only useful if all five fields are right together, and five fields that are each 95% right give only about 77% of claims fully right.** I therefore measured both.

The data also had problems:

- **Clerk values are not perfect: Of** 22 receipt-number differences I checked against the photos, the tool was wrong on 10 and the clerk on 12. Thirteen clerk GSTINs were blank, and CLM-30421 (bill ₹250, clerk typed ₹240) and CLM-30469 (bill ₹335, clerk typed ₹435) show clerk amount typos.
- **Photos and entries:** Six claims have no photo, CLM-31333 is cut off and CLM-31391 is unreadable. Representatives type dates and amounts by hand, some photos are sideways, and the older app saved full phone paths and .jpeg names.
- **Risky bills:** One bill appears on two claims (CLM-30982 repeats CLM-30741), and three bills carry a printed stamp that tries to instruct an automated reviewer into approving them.
- **Cost:** Nilesh will not pay more per photo than a clerk costs, so the savings figure must subtract the model cost.

2. What I Found Out:

On **277 training claims with clerk values,** the tuned tool achieved the following agreement:

Field	Agreement with clerk	After photo checks
Vendor	98.6%	98.9%
Receipt date	96.4%	99.3%
Receipt number	92.1%	96.4%
GSTIN	89.2% (raw 80.5%)	93.1% to about 95.3%
Total amount	98.2% (raw 97.1%)	Up to 100%
All five fields	77.3% (start 69.3%)	Not recomputed

Tool Validation:

By category, all five fields were right **on Parking 95.0%, Fuel 85.5%, Local travel 84.8%, Courier 77.8%, Lodging**

72.2% and Food 66.0%. Food is the weakest.

Tool Validation:

The training figures are optimistic because the rules and prompts were tuned on those claims. I therefore took 30 random test claims and checked every field against the photo myself.

- Vendor, date, receipt number and amount were right on 30 of 30 claims.
- GSTIN was right on 29 of 30 claims (CLM-31478).
- 29 of 30 claims had all five fields right (96.7%). With a sample this small, the true rate could be roughly 83% to 99%.
- The 95% target is met on four fields. It is not claimed for GSTIN or overall.
- The final file has 190 test predictions: 139 approvals and 51 reviews.

A safety check confirms that none of the approvals has a blank main field, a poor photo, low confidence, an amount of ₹2,000 or more, or a bill amount different from the claim.

3. How I Built the Output:

Stage 1: Reading

1. A vision model reads each bill and returns the five required fields, along with a confidence score for each field. The photo is turned upright and resized to 2,000 pixels before it is sent.
2. **Everything printed on a bill is treated as data, never as an instruction to the AI system.**

Stage 2: Repair and Checks

Each improvement was evaluated by measuring the number of cases it helped and the number of cases it negatively affected.

1. GSTIN Repair

The system corrects GSTIN errors using visually similar characters, GSTIN format rules, check-digit validation, and valid bills from the same vendor. This improved GSTIN accuracy from 80.5% to 89.2%, helping 24 cases with zero cases negatively affected.

2. Amount Repair

The system corrects taxi fare amounts when OCR incorrectly reads an extra 7 from the /- symbol. This repair helped 3 cases, with no cases negatively affected. It was used in four approved claims: CLM-30702, CLM-31182, CLM-31305, and CLM-31308.

3. Suspicious GSTINs and Instruction Detection

The system flags the placeholder GSTIN 27ABCDE1234F1ZH, GSTINs that fail check-digit validation, and bills containing instructions for further review. This change moved 16 claims from approval to review, with zero claims moved from review to approval.

4. GSTIN Reread Rule

When a GSTIN fails check-digit validation, the system attempts to read it again to improve accuracy. It also saves previous reading results to avoid paying for processing the same receipt image twice.

Stage 3: Decision

Policy rules, duplicate checks and photo-quality checks determine whether a claim should be approved or reviewed, with a short reason. **Any claim of ₹2,000 or more, any claim with a blank main field, and any claim with a doubtful photo goes to a clerk.**

Stage 4: Service

The **Streamlit dashboard, launched using streamlit run app.py**, shows the claim details, the receipt photo beside the extracted values, the raw and repaired GSTIN, safety checks and downloadable results.The api.py endpoint accepts a photo and claim information, then returns the extracted fields, decision and reasons.

Stage 5: Three Ways to View the Output

The project provides three ways to access and review the results, each designed for a different purpose: **interactive review, application integration, and offline analysis**.

1) Streamlit Dashboard (app.py / dashboard.py)

The dashboard provides an interactive interface for viewing and verifying receipt-reading results. It displays:

- Receipt images alongside extracted values for easy comparison.
- Receipt details, including vendor name, bill date, bill number, GSTIN, and amount.
- Raw and repaired GSTIN values to show how errors were corrected.
- Confidence scores and safety checks to support reliable verification.
- Approval or review decisions, along with the reasons behind them.
- Downloadable results and reports for further analysis.

The dashboard helps users compare the extracted information with the original receipt and understand why a claim was approved or flagged for review.

2) API Endpoint (api.py)

The API allows other applications to use the receipt-reading system without accessing the dashboard directly. It accepts a receipt image and claim information, then returns:

- Extracted receipt details.
- Approval or review decisions.
- Reasons supporting each decision.

This interface makes it possible to integrate the receipt-reading and claim-review process into other applications.

3) Saved Output Files (predictions.csv and output)

The system also saves its results in files for later inspection, reporting, and analysis. These include:

- predictions.csv containing test predictions.
- OCR results and claim decisions.
- Mismatch reports for identifying differences or errors.
- Quality and mapping reports.

- Manual hand-check tables for verification.
- Evaluation reports for reviewing system performance.

These files help track results, evaluate accuracy, and review claim decisions without needing to run the full model again.

4. Project Structure:

I organized the Tarang Expense AI project into separate folders and files so that each component has a clear purpose. This makes the system easier to understand, test, maintain, and extend.

1. source/ – Configuration and AI Setup

This folder contains the common settings and AI model connection.

- config.py – Stores file paths, business rules, image-quality thresholds, and model settings. It reads the API key from the .env file.
- llm_client.py – Connects to the vision model, handles retries and API errors, and responds safely when no API key is available.

2. modules/ – Main Processing Logic

This folder contains the core functions used to process receipt images, validate extracted information, and make claim decisions.

- analysis.py – Inspects datasets, maps receipt photos to claims, and generates analysis reports.
- preprocess.py – Loads receipt photos and checks their orientation, blur, brightness, and contrast.
- ocr.py – Extracts the vendor name, bill date, bill number, GSTIN, and total amount from receipt images.
- validation.py – Repairs extraction errors, validates receipt details, and compares them with the employee's claim.
- gstin_reread.py – Attempts a second GSTIN reading when check-digit validation fails, if this module is used by the running application.
- decision.py – Applies expense policy rules, checks for duplicates and suspicious GSTINs, and decides whether to approve a claim or send it for review.
- evaluation.py – Measures field accuracy, analyses errors, estimates savings, and validates the final prediction file.

3. data/ – Input Data

This folder contains the original datasets and receipt images provided for the project.

- images/ – Contains the 467 original receipt photos, which remain unchanged.
- claims_train.csv – Contains historical claims, typed details, and previous decisions.
- claims_test.csv – Contains the 190 test claims requiring predictions.
- employees.csv – Stores employee details, grades, and vehicle numbers.
- expense-policy.pdf – Contains Tarang's expense policy, version 3.2.
- email-thread.txt – Contains the client's requirements and related email discussions.
- sample_submission.csv – Defines the required format for the final prediction file.

- README.txt – Provides the supplied dataset column definitions.

4. output/ – Generated Results and Reports

This folder stores the outputs produced during receipt processing, validation, and evaluation.

- ocr_results.csv – Stores extracted receipt fields and confidence scores.
- ocr_cache.json – Saves previous receipt-reading results to avoid unnecessary repeated model calls, when used by the application.
- claim_decisions.csv – Records claim decisions, policy checks, duplicate flags, and review reasons.
- ocr_mismatches.csv – Identifies differences between OCR results and employee-submitted claim details.
- ocr_vs_clerk_differences.csv – Compares OCR results with the clerk's typed values for historical claims.
- image_quality_report.csv – Records image quality measurements and quality status.
- receipt_mapping_report.csv – Maps claims to receipt photos and identifies missing or unmatched images.
- check_table.csv – Contains evidence from the manual verification of 30 claims.
- evaluation_report.txt – Summarizes accuracy, error analysis, limitations, and estimated savings.
- dataset_analysis_report.txt – Reports dataset statistics, missing values, duplicates, and data-quality findings.

These files support performance evaluation, error investigation, and manual review of claim decisions.

5. Main Application Files

The main files in the project root connect the processing modules and make the system accessible.

- app.py: Starts the Streamlit application.
- dashboard.py: Displays dashboard metrics, filters, receipt previews, results, and downloadable reports.
- api.py: Provides an API endpoint that accepts a receipt photo and claim information and returns extracted fields, a decision, and the reasons behind it.
- pipeline.py: Runs the complete processing flow, including dataset analysis, image checks, OCR, validation, and claim decisions.
- predictions.csv: Stores the final predictions for all 190 test claims.
- requirements.txt: Lists the Python libraries needed to run the project.
- README.md: Explains the project, setup instructions, execution steps, limitations, and behaviour when no API key is available.
- memo.md: Summarizes the recommendation, estimated savings, and required next steps for Nilesh.
- submission-form.md: Contains the completed submission form.
- .gitignore: Prevents secrets, virtual environments, caches, and excluded files from being added to Git.

6. Streamlit Dashboard: Final Output Interface

I used Streamlit to build a simple, interactive dashboard for reviewing receipt-reading results and claim decisions.

The dashboard has the following features:

1. **Dataset Overview:** Displays total claims, approved claims, claims sent for review, review rate, estimated clerk

time saved, and estimated clerk cost avoided.

2. **Filters and Claim Search:** Allows users to filter claims by claim ID, claim set, category, image status, blur status, and decision.
3. **Image Quality Analysis:** Helps users inspect receipt image quality and identify images that may affect OCR results.
4. **OCR Results:** Displays information extracted from receipt photos and helps users verify the extracted fields.
5. **Mismatch Analysis:** Highlights differences between extracted receipt information and the claim details.
6. **Claim Decisions:** Shows approval or review outcomes and helps users understand the reasons behind each decision.
7. **Reports and Downloads:** Provides access to generated reports for further checking and record-keeping.Instead of requiring users to examine terminal output or open multiple CSV files, the dashboard brings important information together in one place. This makes it easier to identify incorrect readings, investigate suspicious claims, understand review decisions, and evaluate system performance.

7. API Integration

I used FastAPI to make the receipt-reading system accessible to other applications. The API accepts a receipt image and the employee's claim information, processes the receipt, and returns the extracted fields, claim decision, and reasons behind that decision.This allows other applications to use the receipt-processing workflow without requiring users to access the Streamlit dashboard directly.

8. Three Ways to Access the Output

The project provides three ways to view and use the results.

1. **Streamlit Dashboard:** Allows users to interactively review receipt images, extracted values, claim decisions, summary metrics, and reports.
2. **API Endpoint:** Allows other applications to submit receipt images and claim information and receive the processing results.
3. **Saved Output Files:** Stores predictions, OCR results, claim decisions, mismatch reports, image-quality reports, and evaluation reports for later review and analysis.

Together, these three methods support interactive review, application integration, and offline analysis.

Overall Summary

The Tarang Expense AI project combines receipt-image processing, OCR, data repair, policy validation, claim decisions, and evaluation into a single workflow. Its modular structure makes the code easier to maintain, while the Streamlit dashboard, FastAPI endpoint, and saved output files provide three ways to access and review the results.

The system helps users extract receipt details, identify possible errors, apply expense policy checks, and decide whether claims should be approved or sent for manual review. It also provides reports to measure accuracy, investigate mismatches, and understand system performance.

5. Limitation of the AI Analysis:

- **Receipt numbers:** The tool misread 10 bill numbers through dropped or added digits, despite confidence

scores between 0.9 and 1.0. There is no current safeguard to catch all these errors, so a wrong number could be approved.

- **GSTIN:** Accuracy remains below 95% in the reported results. Five test claims have incorrect GSTIN values: CLM-31140, CLM-31158, CLM-31233, CLM-31299 and CLM-31478. All are in review, but the file still holds the wrong values.
- **Vendor identification:** CLM-30527 appears to have read the passenger's name as the vendor, while CLM-30944 may have read a brand instead of the dealer.
- **New vendors:** GSTIN repair requires three valid sightings of a vendor, so new vendors may not benefit from this repair.
- **Instruction stamps:** The system caught the stamps on CLM-30623 and CLM-31401 but did not transcribe the stamp on CLM-30982. That claim is in review for other reasons.
- **Small validation sample**: Only 30 claims were checked manually. Receipt dates were checked against the representative's entered date, not independently against the photos.
- **Missing photos:** Blank fields are correct where photos are missing, and all those claims are in review.
- API dependency: Reading a new photo requires a paid vision-model API call. Without an API key, the dashboard can display saved results, but a new read fails with a clear message.

AI Token Usage and Cost

The project uses a paid GPT mini vision-model API to read receipt photos. Without an API key, the dashboard can display saved results, but new receipt readings require a valid key and available credits. The model name should be verified in config.py.

Actual API Usage (9–16 October 2026)

Metric	Actual usage
API requests	3,034
Tokens used	40,247,791
Total spend	$11.27
Claims processed	473

Cost and Estimated Benefits

- Cost per claim: Approximately $0.024 (₹2.10).
- Monthly model cost: Approximately ₹5,030 for 2,400 claims.
- Estimated monthly staff time freed: 102 hours, worth about ₹24,300.
- Estimated net monthly benefit: ₹19,300.
- Estimated annual benefit: ₹2.3 lakh.

These are estimates, not guaranteed cash savings. They assume the test approval rate continues and exclude some review-related time savings.

Token Usage Improvements

- Saved receipt reads to avoid repeat API costs.
- Resized images to 2,000 pixels.
- Used short prompts and JSON-only output.
- Reread only GSTINs that failed check-digit validation.
- Stopped processing immediately when credits ran out.

AI tools used: Claude for planning, coding, debugging, and checking; a GPT mini model for receipt reading.

6. Conclusion:

The project delivers a receipt-reading and decision pipeline with a dashboard, an API endpoint and saved output files.

Key Results

- Improved GSTIN accuracy from 80.5% to 89.2%.
- Improved all-five-field agreement from 69.3% to 77.3% on the tuned training set.
- Achieved 29 correct claims out of 30 in the independent hand-check.
- Generated 190 test predictions: 139 approvals and 51 reviews.
- Passed the reported safety check and validate_predictions_file() validation.
- Estimated net staff-capacity value of approximately ₹19,300 per month after model cost.
- Provided three ways to access the results: the Streamlit dashboard, API endpoint and saved output files.

Final Recommendation

Start a controlled pilot of auto-approval for claims under ₹2,000 only when:

- The bill amount matches the claim.
- The photo is clear.
- All required fields are present.
- Confidence and validation checks pass.
- No duplicate or suspicious-content flag is raised.

Keep claims of ₹2,000 or more, and all uncertain cases, with the clerks. Do not describe the tool as 95% accurate overall.

What I Need from Nilesh

1. Weekly quality audit: A clerk should check one in every 20 approved claims against the photo to measure the real error rate.
2. Agreement on the reference standard: GSTIN and bill-number accuracy should be judged against the photo, not only against clerk-entered values.
3. Review of suspicious claims: CLM-30982 (possible duplicate bill), CLM-30623 and CLM-31401 (instruction-like stamps). The tool flags these cases but does not determine disciplinary outcomes.

4. Budget and access: Approximately ₹5,000 per month for the model and an API key.

Three min Screen Recording Link:

Screen Recording - Karthika S (Tarang-Pharma-Distribution-expense-receipts)

Github Repo Link:

Github Repositry - Tarang-Pharma-Distribution-expense-receipts - Karthika S

Stream:

tarang-pharma-distribution-expense-receipts · Streamlit